Solution: Rank Transform of an Array

Problem Statement

Given an array `arr` containing integers, replace each element with its `rank` in the array.

- The `rank` is determined based on the `size of the element` compared to others in the array.
- Smaller numbers get a lower rank, and equal numbers share the same rank.
- The ranking starts from 1.

Examples

Example 1:

- **Input**: `[10, 20, 20, 30]`
- **Expected Output**: `[1, 2, 2, 3]`
- **Justification**: 10 is the smallest, so its rank is 1. 20, occurring twice, shares the rank 2. 30, being the largest, gets the rank 3.

Example 2:

- **Input**: `[100, 2, 70, 2]`
- **Expected Output**: `[3, 1, 2, 1]`
- **Justification**: 2, being the smallest, is ranked 1. 70 is next, so it's ranked 2. 100, the largest, is ranked 3.

Example 3:

- **Input**: `[5, 5, 5, 5]`
- **Expected Output**: `[1, 1, 1, 1]`
- **Justification**: All elements are the same, so they all share the same rank, 1.

Solution

To solve this problem, we can leverage sorting and hashmap data structures. The idea is to sort a copy of the original array, which helps in assigning ranks sequentially. We then use a hashmap to store the rank of each unique element from the sorted array. This hashmap acts as a look-up table when we iterate through the original array to replace each element with its corresponding rank. This approach ensures accuracy in ranking and handles duplicate values effectively. It's efficient because sorting and hashmap operations are relatively fast for this kind of task.

Step-by-step Algorithm

- **Create a copy** of the original array and sort it. This sorted array will help in assigning ranks.
- **Initialize a hashmap** to store the rank of each unique element.
- **Iterate through the sorted array**, and for each unique element, assign an increasing rank starting from 1, and store it in the hashmap.
- **Iterate through the original array** and replace each element with its rank found in the hashmap.
- **Return the transformed array** with ranks.

Algorithm Walkthrough

Using the input `[100, 2, 70, 2]`:

1. Create Sorted Copy:

- Input: `[100, 2, 70, 2]`
- Sorted Copy: `[2, 2, 70, 100]`

2. Initialize Hashmap for Ranks:

- Hashmap: `{}`

3. Assign Ranks to Unique Elements:

- Consider `2` (first occurrence), assign rank `1`. Hashmap becomes `{2: 1}`.
- Encounter `2` again, skip as it's already ranked.
- Consider `70`, assign rank `2`. Hashmap becomes `{2: 1, 70: 2}`.
- Consider `100`, assign rank `3`. Hashmap becomes `{2: 1, 70: 2, 100: 3}`.
- **Final Hashmap**: `{2: 1, 70: 2, 100: 3}`
- **Explanation**: Each unique number in the sorted array is assigned a sequential rank starting from 1.

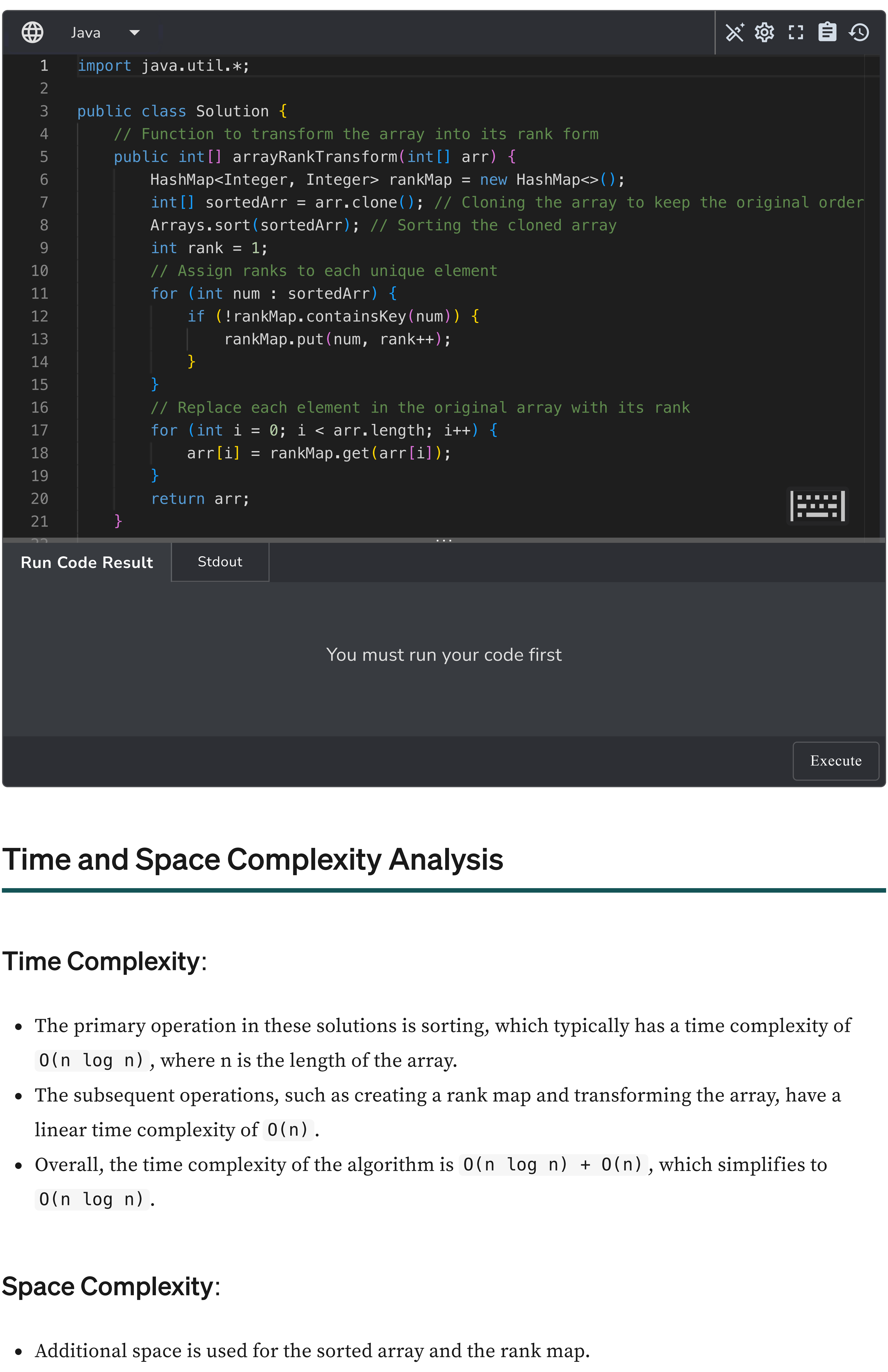
4. Replace Original Array Elements with Ranks:

- Original: `[100, 2, 70, 2]`
- Transformed: `[3, 1, 2, 1]`

5. Return Transformed Array:

- Output: `[3, 1, 2, 1]`

Code



```java
import java.util.*;

public class Solution {
    // Function to transform the array into its rank form
    public int[] arrayRankTransform(int[] arr) {
        HashMap<Integer, Integer> rankMap = new HashMap<>();
        int[] sortedArr = arr.clone(); // Cloning the array to keep the original order
        Arrays.sort(sortedArr); // Sorting the cloned array
        int rank = 1;
        // Assign ranks to each unique element
        for (int num : sortedArr) {
            if (!rankMap.containsKey(num)) {
                rankMap.put(num, rank++);
            }
        }
        // Replace each element in the original array with its rank
        for (int i = 0; i < arr.length; i++) {
            arr[i] = rankMap.get(arr[i]);
        }
        return arr;
    }
```

You must run your code first

Time and Space Complexity Analysis

Time Complexity:

- The primary operation in these solutions is sorting, which typically has a time complexity of `O(n log n)`, where n is the length of the array.
- The subsequent operations, such as creating a rank map and transforming the array, have a linear time complexity of `O(n)`.
- Overall, the time complexity of the algorithm is `O(n log n) + O(n)`, which simplifies to `O(n log n)`.

Space Complexity:

- Additional space is used for the sorted array and the rank map.
- The space complexity for storing the ranks is `O(u)`, where u is the number of unique elements in the array.
- Since the sorted array is a copy of the original array, its space complexity is `O(n)`.
- Therefore, the total space complexity is `O(n) + O(u)`. In the worst case, if all elements are unique, this simplifies to `O(n)`.

← Previous
Rank Transform Of An Array

Next →
Diagonal Traverse (Medium)

☐ Mark as Completed